VOLUME 2

VERIQTA

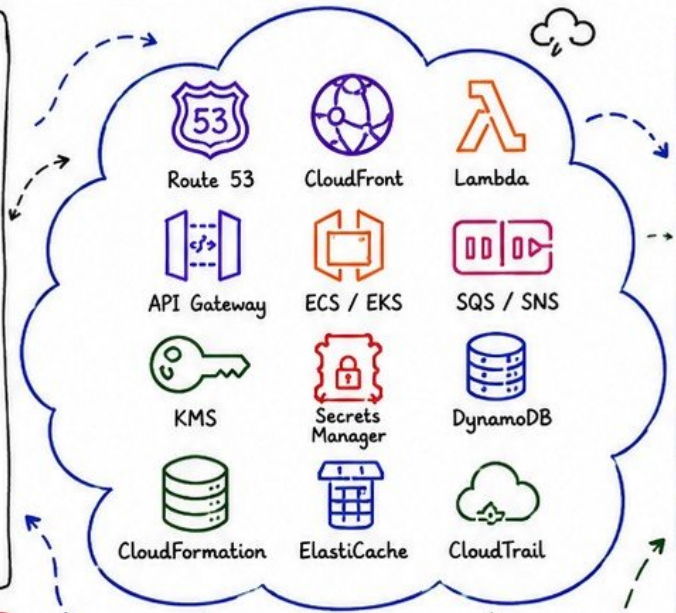
AWS PRODUCTION INCIDENTS

VOLUME 2 – ADVANCED & ENTERPRISE INCIDENTS

REAL INCIDENTS. REAL IMPACT. REAL SOLUTIONS.

WHAT'S INSIDE?

- 15 Advanced AWS Production Incidents
- Serverless, Containers, Messaging & Compliance
- Disaster Recovery Scenarios
- Enterprise Investigation Workflows
- Senior Interview & System Design Prep
- Production Readiness Checklists & Principles



YOU WILL LEARN:

- Deep Dive Troubleshooting of Advanced AWS Services
- How Senior Engineers Investigate & Resolve Complex Incidents
- Design for Scale, Resiliency & High Availability
- Strengthen Security, Compliance & Observability
- Prepare for Disasters & Recover with Confidence

LEVEL UP YOUR SKILLS. SOLVE WHAT OTHERS CAN'T.

YOU - SOLVING PRODUCTION ISSUES LIKE A SENIOR!

PERFECT FOR

- ✓ DevOps Engineers
- ✓ Cloud Engineers
- ✓ SREs & Platform Engineers
- ✓ Solution Architects
- ✓ Anyone running systems in production

> "It's not about avoiding incidents. It's about responding better, recovering faster, and building stronger."

20 PAGES OF

- Real-World Scenarios
- Production-Ready Solutions
- Senior-Level Insights
- Actionable Checklists

LEARN. • TROUBLESHOOT. • RECOVER. • IMPROVE.

BE A PROBLEM SOLVER. BE VALUABLE.

INCIDENT #16

VERIQTA

ROUTE 53 DNS RESOLUTION FAILURE

Users cannot resolve domain names or reach your applications. The root cause is usually DNS misconfiguration, health issues, or routing failure.

AT A GLANCE

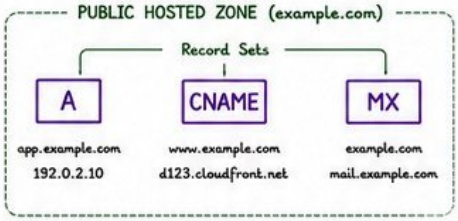
- DNS is the first step
- Fix DNS = Fix Reachability
- Check the basics first
- Validate & test from outside
- Use health checks & failover

1 DNS TROUBLESHOOTING WORKFLOW

1. **Confirm the issue** — Is it global or specific to a region/ISP?
2. **Test name resolution** — dig / nslookup / whatsmydns.net
3. **Check Route 53 health** — Health checks, status of resources
4. **Validate hosted zone** — Records, TTL, routing policy, aliases
5. **Check resolver & network** — VPC resolver, security groups, NACLs
6. **Review recent changes** — Record updates, TTL changes, failover configuration
7. **Monitor & confirm fix** — CloudWatch, synthetic checks, real user tests

2 HOSTED ZONES BASICS

- Hosted Zone is a container for DNS records.
- Public Hosted Zone → Internet-facing domains
- Private Hosted Zone → Internal domains (VPC only)
- Each domain (example.com) must have a single public hosted zone.

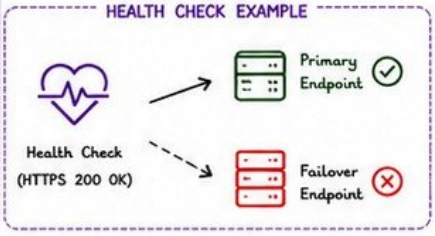


BEST PRACTICES

- Use meaningful TTL (60s – 300s for dynamic apps)
- Use Alias records for AWS resources
- Enable DNSSEC for added security

3 HEALTH CHECKS

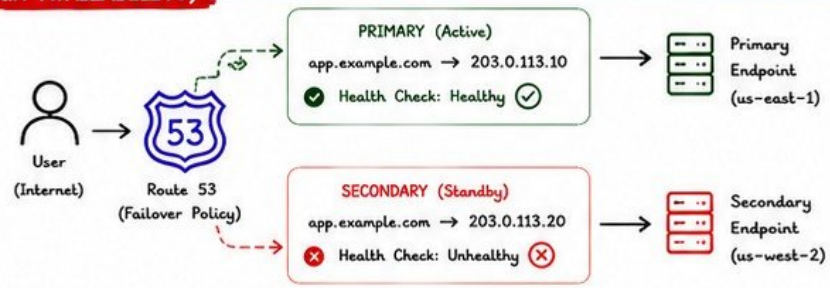
- Health checks monitor the health of your resources.
- Can check: HTTP, HTTPS, TCP, or calculated health.
- Used by failover routing policies to make smart routing decisions.
- Global health checks run from multiple AWS regions.



If primary fails, Route 53 routes to the healthy endpoint.

4 FAILOVER ROUTING (HIGH AVAILABILITY)

- Route 53 can automatically route traffic to healthy endpoints.
- Use Health Checks + Failover Routing Policy.
- Two record sets:
  - PRIMARY → Active under normal conditions
  - SECONDARY → Active when primary fails



SUPPORTED POLICIES

- Simple Routing (Default)
- Weighted
- Latency
- Geolocation
- Geoproximity
- Multi-Value Answer
- Failover

5 COMMON CAUSES

- Incorrect or missing DNS records
- Wrong TTL causing old caching
- Health checks misconfigured
- Endpoint/Destination unreachable
- VPC DNS resolver issues
- Security groups blocking traffic
- Recent DNS changes not propagated
- ISP or network DNS issues (external)

6 USEFUL COMMANDS

dig example.com	Query DNS records
dig @1.1.1.1 example.com	Test from specific DNS server
nslookup example.com	Check DNS resolution
whatsmydns.net	Global DNS propagation check
aws route53 list-hosted-zones	List hosted zones
aws route53 list-resource-record-sets	List DNS records
curl -I https://example.com	Test endpoint health

7 MONITORING & ALERTING

- CloudWatch Alarms for Health Checks and DNS query volumes.
- Route 53 Resolver Query Logs to analyze DNS traffic.
- Synthetic Canary (CloudWatch Synthetics) to test end-to-end.
- Create alarms for failover events (Health Check status changes).

KEY TAKEAWAYS

- ✓ DNS is the foundation of everything.
- ✓ Always test from outside your network.
- ✓ Use health checks + failover for high availability.
- ✓ Monitor, alert & validate after every change.

> If users can't resolve it, they can't reach it.
> Fix DNS. Restore Trust.

REMEMBER

- ☐ Small DNS misconfigurations can cause BIG outages.
- ☐ Prevention + Monitoring = Production Stability.

INCIDENT #17

VERIQTA

CLOUDFRONT SERVING STALE OR MISSING CONTENT

Users see outdated content or 404/403 errors.
The issue is usually caching, origin, or distribution configuration.

AT A GLANCE

- CloudFront is a global cache
- Cache can serve old content
- Origin issues = content missing
- Invalidation fixes stale content
- Edge locations improve performance

1 CACHE BEHAVIOR

- CloudFront caches content based on:
  - Default TTL (min, default, max)
  - Cache-Control headers
  - Expires headers
  - Query strings / Cookies / Headers (cache key)
- Misconfigured TTL or cache policy can cause stale content.

2 ORIGIN FAILURES

- If CloudFront can't get content from origin, users get 5xx/404.
- Common causes:
  - Origin down or unreachable
  - Wrong origin domain/port
  - Permission denied (S3 403)
  - SSL/TLS errors
  - Origin Shield misconfigurations

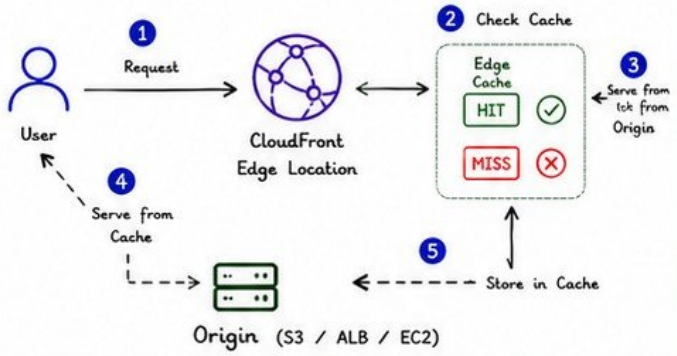
3 CACHE INVALIDATION

- CloudFront caches until TTL expires unless invalidated.
- Use invalidations to purge old objects immediately.
- Best for:
  - New releases
  - Hotfixes
  - Updated images/CSS/JS
  - Removed or renamed files

4 EDGE LOCATIONS

- CloudFront serves from 400+ Edge Locations (PoPs).
- Content may differ across regions due to:
  - Propagation delay
  - Cache not yet updated
  - Regional origin issues

5 HOW CLOUDFRONT WORKS (REQUEST FLOW)



6 COMMON ISSUES & SYMPTOMS

ISSUE	SYMPTOMS	POSSIBLE CAUSE
Stale Content	Old version still shown	Long TTL, no invalidation
404 Not Found	File missing	Origin path wrong, object deleted
403 Forbidden	Access denied	S3 bucket policy / OAC / OAI
5xx Errors	502/503/504	Origin down or timeout
Different content by region	Some users see old, others new	Edge cache not refreshed yet

7 TROUBLESHOOTING STEPS

1. Check URL in browser (hard refresh / incognito).
2. Verify CloudFront distribution status.
3. Check cache behavior settings (TTL, headers, cookies).
4. Review CloudFront logs (standard or real-time).
5. Test origin directly (bypass CloudFront).
6. Check origin health (S3 website/ALB/EC2).
7. Check invalidations list and status.
8. Confirm SSL/TLS certificates.

8 CACHE INVALIDATION EXAMPLE (AWS CLI)

```
aws cloudfront create-invalidation \
  --distribution-id E123ABC456DEF \
  --paths "/*"
```

Invalidation Statuses

In Progress | Completed | Failed

Use targeted paths instead of "/*" to reduce cost and speed up.

9 BEST PRACTICES

- Use appropriate TTL values.
- Use versioned file names (ex: app.v2.js).
- Enable compression (gzip/brotli).
- Use Origin Shield for high request volumes.
- Enable real-time logs for critical distributions.
- Automate invalidations in CI/CD pipeline.
- Monitor 4xx/5xx and cache hit ratio.

KEY TAKEAWAYS

CloudFront is fast because it caches.
Caching is powerful but can cause stale content.
Know when to invalidate and how to test your origin.
Monitor, automate, and optimize your distributions.

> Good caching makes your app fast.
> Smart invalidation keeps it correct.
> Balance speed and freshness.

REMEMBER

- [ ] Cache is your friend, but so is invalidation.
- [ ] Happy users see the right content.

INCIDENT #18

VERIQTA

LAMBDA FUNCTION TIMING OUT

Users experience errors, long delays or API Gateway 502/504.
Root cause is usually slow code, dependencies, or misconfiguration.

AT A GLANCE

- Lambda timeout = function not completing in allowed time
- Check code, memory, dependencies
- Cold starts can add extra latency
- Optimize, configure, monitor

1 COLD STARTS

Cold starts happen when a new execution environment is created.

Causes:

- First request after inactivity
- Scaling to zero
- Large deployment package
- VPC + ENI creation

REDUCE COLD STARTS

- Keep functions warm (Provisioned Concurrency)
- Use smaller packages
- Avoid heavy global imports
- Optimize initialization code

2 MEMORY ALLOCATION

More memory = more CPU power.

Insufficient memory can slow down your function and cause timeouts.

MEMORY	CPU ALLOCATED
128 MB	~7%
512 MB	~29%
1024 MB	~59%
3008 MB	~176%
10240 MB	~600%

BEST PRACTICES

- Start with 512MB – 1024MB
- Increase memory to get more CPU
- Test and find the sweet spot
- Use AWS Lambda Power Tuning

3 TIMEOUT CONFIGURATION

Function exceeds the configured time limit.

- Default timeout: 3 seconds
- Max timeout: 900 seconds (15 min)
- Set timeout > expected duration
- Always leave buffer (10–20%)

EXAMPLE

```
{
  "FunctionName": "process-data",
  "Timeout": 60,   // seconds
  "MemorySize": 1024
}
```

Set based on real metrics, not guessing!

4 DEPENDENCY BOTTLENECKS

External calls or resources can cause delays.

Common bottlenecks:

- Slow database queries
- External APIs
- S3 operations (large files)
- Network latency (VPC, NAT)
- Throttled downstream services

SOLUTIONS

- Optimize queries & add indexes
- Use connection pooling
- Increase timeouts for external calls
- Add retries with exponential backoff
- Cache or batch requests

5 HOW LAMBDA EXECUTES (HIGH LEVEL FLOW)

Timeout occurs if Lambda does not return a response within the configured time limit.

6 COMMON TIMEOUT SYMPTOMS

SYMPTOM	POSSIBLE CAUSE	WHAT TO CHECK
Task timed out after X.XX seconds	Function exceeded timeout	Increase timeout or optimize code
High duration in logs	Slow code or dependencies	CloudWatch Logs, X-Ray traces
API Gateway 504/502	Lambda took too long	Check Lambda + downstream latency
Throttles + timeouts	Insufficient concurrency	Check concurrency limits & usage
Timeout only on first call	Cold start delay	Provisioned Concurrency / optimize init

7 TROUBLESHOOTING CHECKLIST

1. Check CloudWatch Logs (Duration, Billed Duration, Timeout errors)
2. Check function timeout setting
3. Review memory allocation and CPU performance
4. Analyze code for inefficient logic or blocking calls
5. Check external dependencies (latency, errors, throttling)
6. Review VPC configuration (ENI setup can add seconds)
7. Check cold start impact (Provisioned Concurrency?)
8. Use AWS X-Ray to trace and find slow segments

8 OPTIMIZATION TIPS

- Use async patterns (SQS, EventBridge) for long tasks (> 15 min use Step Functions or Fargate)
- Increase memory to improve performance
- Cache data (DynamoDB, ElastiCache, S3)
- Minimize package size and dependencies
- Reuse connections (DB, HTTP clients)
- Use Provisioned Concurrency for latency-critical apps
- Monitor and set alarms on Duration & Errors

CLOUDWATCH METRICS TO WATCH

- Duration
- Billed Duration
- Throttles
- Errors
- Concurrent Executions
- Init Duration (Cold Start)
- Max Memory Used

KEY TAKEAWAYS

- ✓ Lambda timeout is almost always a performance or configuration issue.
- ✓ More memory = more CPU = faster execution.
- ✓ Understand your dependencies and optimize end-to-end.
- ✓ Measure, monitor, and iterate!

> Fast code. Right config.
> Healthy dependencies.
> That's how timeouts disappear.

REMEMBER

- ☐ Never guess — measure.
- ☐ Think beyond your code.
- ☐ Optimize, Automate, Monitor.

INCIDENT #19

VERIQTA

API GATEWAY RETURNING 502 / 504 ERRORS

Users see 502 Bad Gateway or 504 Gateway Timeout.
Root cause is usually integration failures, backend issues, or timeouts.

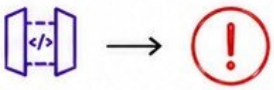
AT A GLANCE

- 502 = Bad Gateway (integration failed)
- 504 = Gateway Timeout (took too long)
- Check integration, backend & timeout
- Lambda, HTTP, VPC links can fail
- Use logs, metrics & tracing

1 WHAT DO 502 / 504 MEAN?

API Gateway acts as a proxy.
If it can't get a valid response from the backend:

- 502 Bad Gateway = Invalid response or connection failure
- 504 Gateway Timeout = No response within timeout



2 COMMON INTEGRATION FAILURES

- Lambda function errors or timeouts
- Incorrect Lambda permissions / resource policy
- HTTP endpoint down or unreachable
- DNS resolution failure
- VPC Link down (private integrations)
- Security group / NACL blocking traffic
- Backend returns invalid HTTP response
- Mapping templates / payload issues

3 LAMBDA BACKEND ISSUES

- Function timeout > API Gateway timeout
- Lambda returns errors (handled/unhandled)
- Throttling / Concurrency limits reached
- Cold start causing delays (504)
- Insufficient memory leads to slow processing

CHECKLIST

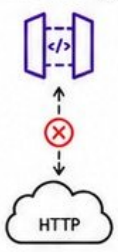
- [ ] Check Lambda CloudWatch Logs
- [ ] Check function timeout setting
- [ ] Increase memory for better performance
- [ ] Monitor duration & errors

4 HTTP INTEGRATION ISSUES

- Backend HTTP service not reachable
- Incorrect endpoint or path
- Backend returning 4xx/5xx
- SSL/TLS handshake failure
- Slow backend response

TIPS

- Test endpoint using curl / Postman
- Verify security groups & NACLs
- Check service health & status



5 API GATEWAY TIMEOUT CONFIGURATION

Default Integration Timeout: 29 seconds
Max Timeout: 29 seconds (REST APIs)
If backend takes longer → 504 Gateway Timeout

BEST PRACTICES

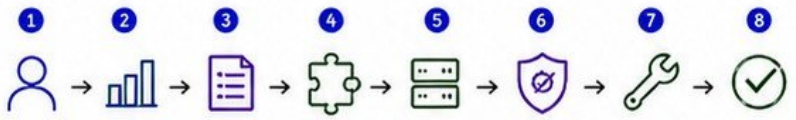
- Optimize backend response time
- Use caching where possible
- Increase Lambda memory
- Move long-running tasks to async (SQS, Step Functions)

6 GATEWAY LOGS & METRICS

ENABLE & CHECK

- Access Logs → See request/response
- Execution Logs → Debug integration
- CloudWatch Metrics to monitor:
  - 5XXError
  - IntegrationLatency
  - Latency
  - Count

7 TROUBLESHOOTING WORKFLOW



1. Reproduce the error
2. Check CloudWatch Metrics
3. Check Execution Logs
4. Verify Integration Type
5. Test Backend (Lambda / HTTP) Directly
6. Check VPC, Security Groups, DNS
7. Fix issue & redeploy
8. Monitor & validate

8 COMMON CAUSES QUICK REFERENCE

STATUS	CAUSE	WHAT TO CHECK
502 Bad Gateway	Invalid response / Connection failure	Integration response, Mapping templates, backend, VPC Link, SG, DNS
504 Gateway Timeout	No response within timeout	Backend performance, Lambda timeout, network latency

KEY TAKEAWAYS

- ✓ 502 = Integration failed. 504 = Timed out.
- ✓ Always validate backend health & connectivity.
- ✓ Logs + Metrics + Direct tests = Fast root cause.
- ✓ Optimize timeout, memory, and backend performance.

> The gateway is only as strong as the system behind it.
> Fix the backend, fix the gateway.

REMEMBER

- [ ] Logs don't lie.
- [ ] Metrics show the pattern.
- [ ] Test like a user.
- [ ] Monitor like a pro.

INCIDENT #20

VERIQTA

ECS TASKS CONSTANTLY RESTARTING

Users see service instability, 5xx errors or tasks that keep stopping and starting.
Root cause is usually bad resource limits, crashes, or misconfiguration.

AT A GLANCE

- ECS tasks restart when the container exits unexpectedly
- Check task definition, resources, logs & health checks
- Common causes: OOM, app crash, bad config, dependency issues
- Fix root cause & ensure service recovery

1 WHAT DOES IT MEAN?

ECS task goes to STOPPED and starts again.

You may see:

- High restart count
- Tasks in PENDING → RUNNING → STOPPED loop
- Service never stable
- ALB targets unhealthy

2 TASK DEFINITIONS

Task definition controls how containers run.

Check:

- Image & tag
- Port mappings
- Environment variables
- Health check command
- Log configuration
- Resource limits (CPU/Memory)

3 CPU & MEMORY LIMITS

Insufficient resources cause OOM (Out Of Memory) kills or throttling.

Best practices:

- Set both CPU and memory
- Reserve enough memory
- Monitor usage
- Understand Fargate scaling
- Avoid setting limits too low

EXAMPLE (FARGATE)

Task Size	CPU	Memory
0.25 vCPU	0.25	512 MB
0.5 vCPU	0.5	1 GB
1 vCPU	1	2 GB
2 vCPU	2	4 GB

4 CONTAINER LOGS

Logs show why the container is exiting.

Check for:

- Application errors / stack trace
- Missing dependencies
- Port already in use
- Configuration issues
- Out of memory (OOMKilled)
- Permission denied / errors

5 COMMON RESTART CAUSES

- Out of memory (exit code 137)
- Application crash / unhandled exception
- Health check failing
- Crash loop due to bad configuration
- Dependency / service not reachable
- Port conflict
- Image pull failure
- Secrets / Env / Permission issues
- Insufficient CPU (throttling)

OOMKilled is the #1 cause for Fargate task restarts.

6 SERVICE RECOVERY

ECS Service maintains desired count and replaces failed tasks.

Make sure you have:

- Desired count > 0
- Deployment circuit breaker (on)
- Health check configured
- Rolling update (min/max healthy %)
- Proper retry/backoff strategy

7 TROUBLESHOOTING STEPS

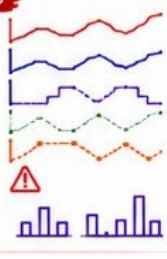
1. Check service events (ECS Console)
2. Check task stopped reason
3. Review container logs (CloudWatch Logs)
4. Check CPU/Memory metrics
5. Verify task definition settings
6. Test image locally
7. Check Health checks
8. Confirm dependencies (DB, API, etc.)

8 USEFUL AWS CLI COMMANDS

```
aws ecs describe-services --cluster my-cluster --services my-service                    # Check service status
aws ecs describe-tasks --cluster my-cluster --tasks <task-id>                           # Check task details
aws ecs describe-task-definition --task-definition <family>                             # Check task definition
aws ecs logs --cluster my-cluster --follow --task <task-id>                             # Stream container logs
aws logs describe-log-streams --log-group-name <log-group> --order-by LastEventTime    # List log streams
aws logs get-log-events --log-group-name <log-group> --log-stream-name <stream> --limit 50 # View log events
aws ecs update-service --cluster my-cluster --service my-service --force-new-deployment # Redeploy service
```

9 MONITORING METRICS

- CPUUtilization
- MemoryUtilization
- TaskCount
- RunningTaskCount
- PendingTaskCount
- ServiceDeploymentFailed
- TaskRestarts (custom metric)



KEY TAKEAWAYS

- ✓ Most restarts are resource or configuration issues.
- ✓ Right-size CPU/Memory.
- ✓ Check logs first, metrics second.
- ✓ Enable auto recovery and health checks.
- ✓ Monitor and fix root cause, not the symptom.

"Stable containers. Healthy tasks. Resilient services. That's production ready.

REMEMBER

- ☐ Logs don't lie.
- ☐ Metrics show the trend.
- ☐ Automate recovery.
- ☐ Design for failure.

INCIDENT #21

VERIQTA

EKS WORKER NODES NOT JOINING THE CLUSTER

Nodes launch successfully but remain in NotReady state or never show up in the cluster.
Root cause is usually IAM, bootstrap, networking, or security.

AT A GLANCE

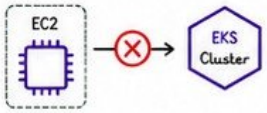
- Worker nodes fail to register with EKS
- Check IAM role, bootstrap, network, and security groups
- Use logs and describe command to identify the failure point
- Fix the root cause and recycle nodes

1 WHAT DOES IT MEAN?

EKS worker node instances are running, but they do not join the cluster.

Symptoms:

- Node not listed in kubectl get nodes
- Node is in NotReady state
- Node joins then disappears
- Pods stuck in Pending



Node not registered

2 IAM ROLES

Missing or incorrect IAM role is the #1 cause.

Check:

- Instance profile attached
- Role has AmazonEKSWorkerNodePolicy
- Role has AmazonEC2ContainerRegistryReadOnly
- Role has AmazonEKS_CNI_Policy
- Role trust relationship with ec2.amazonaws.com

3 BOOTSTRAP ISSUES

The node fails during kubelet bootstrap or cannot communicate with the API Server.

Check:

- User data / launch template
- /etc/eks/bootstrap.sh command
- Cluster name, endpoint, CA cert
- Kubelet service status
- Time sync (NTP/chrony)

Example bootstrap command

```
/etc/eks/bootstrap.sh \
--apiserver-endpoint <endpoint> \
--b64-cluster-ca <cluster-ca> \
--kubelet-extra-args "--node-labels=
role=worker\""
```

4 NETWORKING

Nodes must reach the EKS API Server and required endpoints.

Check:

- Private subnet route to NAT/IGW
- Security group outbound rules
- Node can reach EKS endpoint
- DNS resolution (CoreDNS/Route 53)
- VPC endpoints (if using private API)

5 SECURITY GROUPS

Incorrect SG rules block communication.

Node SG must allow OUTBOUND:

- TCP 443 to EKS API endpoint
- TCP 10250 to control plane (self)
- All outbound (recommended)

Control Plane SG must allow INBOUND:

- TCP 443 from Node SG (API access)
- TCP 10250 from Node SG (kubelet)

6 COMMON FAILURE POINTS

FAILURE POINT	WHAT TO CHECK
IAM Role	Instance profile, policies, trust relationship
Bootstrap	User data, cluster name, CA, endpoint, kubelet logs
Networking	Routes, NAT, DNS, security groups, VPC endpoints
Security Groups	Outbound from node, inbound to control plane
Time Sync	NTP/chrony running, correct time
CNI Plugin	AWS VPC CNI installed and running
Kubelet	Service status, logs, config

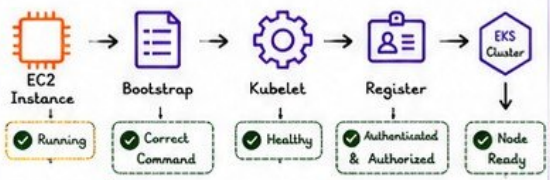
7 TROUBLESHOOTING STEPS

1. Check node in EC2 console (running?)
2. Verify IAM role and instance profile
3. Check user data / bootstrap command
4. Check node logs: /var/log/cloud-init-output.log
5. Check kubelet logs: journalctl -u kubelet -xe
6. Test connectivity to API endpoint (curl 443)
7. Check security group inbound/outbound rules
8. Verify DNS resolution (nslookup <endpoint>)
9. Check VPC endpoints (if using private access)
10. Recycle node or launch new one

8 USEFUL COMMANDS (ON THE NODE)

```
cat /etc/eks/bootstrap.sh
systemctl status kubelet
journalctl -u kubelet --no-pager -xe
curl -v https://$API_SERVER/healthz
nslookup $API_SERVER
ip route
aws eks describe-cluster --name <cluster>
aws ec2 describe-instances --instance-ids <id> \
--query "Reservations[0].Instances[0].IamInstanceProfile"
```

9 NODE JOIN FLOW



10 QUICK CHECKLIST

- [ ] IAM Role attached with correct policies
- [ ] Bootstrap command correct
- [ ] API endpoint reachable (443)
- [ ] Security groups allow traffic
- [ ] NAT/IGW in private subnets
- [ ] DNS resolution working
- [ ] Kubelet service running
- [ ] CNI plugin installed

KEY TAKEAWAYS

- ✓ IAM, bootstrap, networking, and security are the top causes.
- ✓ Always check logs and connectivity before making changes.
- ✓ Use a systematic approach to validate each layer.
- ✓ A healthy node = stable cluster.

> A node that can't communicate cannot contribute.
> Fix the path, then the cluster will do the rest.

REMEMBER

- [ ] Check assumptions.
- [ ] Verify access.
- [ ] Observe logs.
- [ ] Automate recovery.

INCIDENT #22

VERIQTA

SECRETS MANAGER RETRIEVAL FAILURE

Applications fail to retrieve secrets from AWS Secrets Manager. Root cause is usually permission issues, KMS encryption, rotation problems, or integration misconfiguration.

AT A GLANCE

- Verify IAM permissions and resource policy
- Check KMS key access and encryption
- Validate secret rotation and version
- Ensure correct integration in application
- Use logs and metrics to identify issues

1 WHAT DOES IT MEAN?

The application cannot retrieve a secret or gets AccessDenied, ResourceNotFound, or DecryptionFailure.

Common symptoms:

- App errors on startup
- DB connection failures
- 4xx / 5xx errors in services
- Secrets Manager exceptions
- Tasks or pods crash looping

App → Secrets Manager

2 IAM PERMISSIONS

Incorrect or missing IAM permissions is the #1 cause.

Check:

- secretsmanager:GetSecretValue
- secretsmanager:DescribeSecret
- secretsmanager:ListSecretVersionIds
- Resource policy (if cross-account)
- Condition keys (aws:SourceVpce, aws:SourceArn, etc.)

Role / User → Secrets Manager

3 KMS ENCRYPTION ISSUES

Secrets are encrypted with a KMS key. If access is denied, decryption fails.

Check:

- Caller has kms:Decrypt permission
- Key policy allows the principal
- Grants are valid (not revoked)
- Using the correct region
- Cross-account key access configured

FLOW: App/Role → KMS Key → Secret

4 SECRET ROTATION PROBLEMS

Rotation failures can cause outdated or invalid secrets.

Check:

- Rotation Lambda errors
- Rotation function permissions
- Networking to target service
- Staging labels: AWSCURRENT, AWSPREVIOUS
- Verify new version is valid

5 APPLICATION INTEGRATION

Wrong secret name, region, or endpoint can break retrieval.

Check:

- Secret name / ARN correct
- Using correct AWS region
- VPC endpoint / NAT access
- SDK / CLI version
- Caching / connection reuse
- Environment variables / path

6 COMMON ERROR MESSAGES

AccessDeniedException
IAM permission or resource policy issue

DecryptionFailureException
KMS key access problem

ResourceNotFoundException
Secret name / ARN does not exist

ThrottlingException
Rate limit exceeded

InternalServiceErrorException
AWS internal issue

TIP: Always check CloudTrail for the exact error and caller identity.

7 TROUBLESHOOTING STEPS

1. Confirm secret name / ARN / region
2. Verify IAM permissions (identity + resource policy)
3. Check KMS key permissions and key policy
4. Test access using AWS CLI

```
aws secretsmanager get-secret-value \
  --secret-id <name-or-arn>
```

5. Check VPC endpoint / network connectivity
6. Review CloudTrail for denied events
7. Validate rotation Lambda logs (if enabled)
8. Check secret version stage (AWSCURRENT)
9. Test application integration locally
10. Fix and re-test retrieval

8 USEFUL AWS CLI COMMANDS

```
# Get secret value
aws secretsmanager get-secret-value \
--secret-id my-secret

# Describe secret
aws secretsmanager describe-secret \
--secret-id my-secret

# List versions
aws secretsmanager list-secret-version-ids \
--secret-id my-secret

# Check rotation status
aws secretsmanager describe-secret \
--secret-id my-secret \
--query "RotationEnabled"
```

9 MONITORING & ALERTING

Enable metrics and alarms.

Key Metrics:

- Secret retrieval latency
- 4xx / 5xx errors
- Throttles
- Rotation success/failure
- API calls

CloudWatch Alarms:

- High 4xx/5xx rate
- RotationFailed > 0
- Throttles > threshold

10 RETRIEVAL FLOW (SUCCESS PATH)

Application / Service → IAM Role → Secrets Manager → KMS Key (Decrypt) → Secret Value

Best Practices

- Use least privilege IAM policies
- Enable automatic rotation
- Use customer managed KMS keys
- Store only what you need
- Cache secrets securely
- Monitor & alert on failures

KEY TAKEAWAYS

- ✓ Permissions, encryption, rotation, and integration are the top causes.
- ✓ Check CloudTrail first.
- ✓ Validate with AWS CLI.
- ✓ Monitor and automate secret rotation.

> Secrets protect your application.
> Secure access keeps your business safe.
> Automate. Monitor. Rotate.

REMEMBER

- ☐ Least privilege IAM
- ☐ Correct region & networking
- ☐ Correct KMS key access
- ☐ Test integration
- ☐ Validate rotation
- ☐ Monitor and alert

COMMON LOG SOURCES

- CloudTrail (API events)
- Secrets Manager Console Logs
- Lambda Logs (rotation)
- Application Logs

INCIDENT #23

VERIQTA

CLOUDFORMATION STACK ROLLBACK FAILED

CloudFormation stack creation/update fails and rollback also fails, leaving stack in ROLLBACK_FAILED state. Root cause is usually failed resources, dependencies, or manual changes (drift).

AT A GLANCE

- Understand failure and rollback reason
- Check failed resources and dependencies
- Detect and fix drift or conflicts
- Choose the right recovery strategy
- Clean up and prevent recurrence

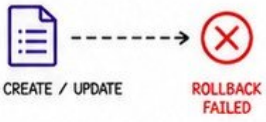
1 WHAT DOES IT MEAN?

CloudFormation cannot roll back the stack to the previous stable state after a failure.

Stack Status: ROLLBACK_FAILED

Common causes:

- Resource creation/update failed
- Rollback resources also failed
- Dependency constraints
- Manual changes (drift)
- Missing permissions or limits



2 DRIFT DETECTION

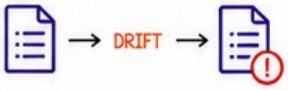
Manual changes outside CloudFormation can break rollback.

Check:

- Properties changed manually
- Resources deleted outside stack
- New resources added manually
- Out-of-sync resource values

CLI COMMAND

```
aws cloudformation detect-stack-drift \
--stack-name my-stack
```



3 FAILED RESOURCES

Identify which resources caused the failure.

Check:

- Events tab in CloudFormation
- Look for status: CREATE_FAILED or UPDATE_ROLLBACK_FAILED
- Note the logical ID and reason
- Check dependent resources

EXAMPLE EVENT

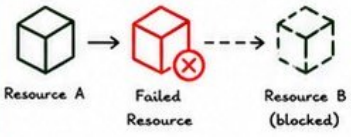
```
{
  "ResourceStatus": "CREATE_FAILED",
  "LogicalResourceId": "MyBucket",
  "ResourceStatusReason":
    "Bucket name already exists"
}
```

4 DEPENDENCY CONFLICTS

Resources depend on each other.
If one fails, others cannot roll back.

Examples:

- S3 Bucket in use by policy
- VPC in use by ENI / NAT / IGW
- IAM Role attached to resources
- Custom resource Lambda failure



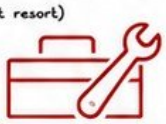
5 RECOVERY STRATEGIES

1. Fix the root cause of failed resource
2. Update template if needed
4. Continue update rollback (if possible)
4. Skip failed resources (if safe)
5. Delete and recreate stack (last resort)

WHEN TO RECREATE?

Use when:

- Many resources failed
- Dependencies are heavily broken
- Stack is in inconsistent state



6 RECOVERY STEPS (PRACTICAL FLOW)

Investigate
Check events, failed resources & reasons

↓

Fix
Correct issues in template, permissions, or config

↓

Retry
Continue update rollback or update stack

↓

Recreate (if needed)
Delete stack and recreate from template

7 USEFUL CLI COMMANDS

```
# Describe stack events
aws cloudformation describe-stack-events \
--stack-name my-stack
# Detect drift
aws cloudformation detect-stack-drift \
--stack-name my-stack
# Continue update rollback
aws cloudformation continue-update-rollback \
--stack-name my-stack --resources-to-skip
LogicalResourceId
# Delete stack
aws cloudformation delete-stack \
--stack-name my-stack
```

8 PREVENTION BEST PRACTICES

- Use meaningful logical IDs and stack structure
- Test templates with change sets
- Enable termination protection for critical stacks
- Apply least privilege IAM for CloudFormation
- Monitor stack events and set up alerts
- Use drift detection regularly

9 COMMON ERROR MESSAGES

ERROR MESSAGE	WHAT IT MEANS
The following resource(s) failed to create	Resource creation failed
UPDATE_ROLLBACK_FAILED	Rollback also failed
Resource handler returned message: Invalid request	Invalid parameter or config
Circular dependency between resources	Dependency loop in template
Insufficient capabilities	Missing IAM capabilities

TIP

Always read the first failure in the events list. That's your root cause!

KEY TAKEAWAYS

- ✓ Rollback fails when resources can't be reverted.
- ✓ Find the root cause, fix it, and choose a recovery path.
- ✓ Recreate stack only when necessary.
- ✓ Prevention is cheaper than recovery.

> Infrastructure is code, but recovery is strategy. Plan. Observe. Recover.

REMEMBER

- ☐ Events don't lie.
- ☐ Drift causes pain.
- ☐ Dependencies matter.
- ☐ Automate recovery.

INCIDENT #24

VERIQTA

SQS QUEUE BACKLOG GROWING RAPIDLY

Messages pile up in the queue and are not processed fast enough, causing delays, timeouts, or downstream impact. Root cause is usually imbalance, visibility, or consumer scaling.

AT A GLANCE

- Identify why messages accumulate
- Tune visibility timeout and retries
- Use DLQ for poison messages
- Scale consumers to match load
- Monitor and set smart alerts

1 WHAT DOES IT MEAN?

Messages are added to the SQS queue faster than they are being processed.

You may see:

- ApproximateAgeOfOldestMessage increasing
- ApproximateNumberOfMessages high
- Consumers running but backlog not reducing
- Delayed processing and timeouts

Too many in → Too few out = Backlog grows

2 PRODUCER vs CONSUMER IMBALANCE

The most common cause of backlog.

Check:

- Producers sending too many messages
- Consumers not keeping up
- Slow or failing downstream services
- Batch size too small
- Inefficient message processing

High In — Low Out

3 VISIBILITY TIMEOUT ISSUES

If visibility timeout is too short, messages may reappear before processing completes.

Check:

- Visibility timeout < processing time
- Messages become visible again
- Duplicate processing occurs
- Change too frequently causes retry storms

MESSAGE LIFECYCLE

Timeout too short = message reappears

4 DEAD LETTER QUEUES (DLQ)

Poison or unprocessable messages should go to DLQ.

Check:

- DLQ is configured
- Redrive policy set correctly
- Messages exceeding max receives move to DLQ
- Monitor DLQ size and alarms
- Fix root cause and reprocess

5 SCALING CONSUMERS

Add more consumers or increase capacity to match workload.

Options:

- Scale EC2/ECS/Lambda consumers
- Increase concurrency (Lambda)
- Use Auto Scaling policies
- Optimize batch size
- Parallelize message processing

Scale Out = Clear Backlog

6 COMMON CAUSES

- Producer sending spikes
- Consumer instances down
- Slow downstream dependencies
- Visibility timeout too low
- Large messages/ big payloads
- Code errors causing retries
- No DLQ for failed messages

Backlog now = Incident
Backlog later = Outage

7 TROUBLESHOOTING STEPS

1. Check ApproximateAgeOfOldestMessage
2. Compare In-flight vs visible messages
3. Review CloudWatch metrics
4. Check consumer logs and errors
5. Verify visibility timeout setting
6. Inspect DLQ for failed messages
7. Validate downstream service health
8. Check scaling and instance health
9. Test message processing manually
10. Reprocess DLQ messages after fix

8 USEFUL CLOUDWATCH METRICS

- ApproximateNumberOfMessagesVisible
- ApproximateNumberOfMessagesNotVisible
- ApproximateNumberOfMessagesDelayed
- ApproximateAgeOfOldestMessage
- NumberOfMessagesSent
- NumberOfMessagesReceived
- NumberOfMessagesDeleted
- NumberOfMessagesMovedToDLQ

9 USEFUL AWS CLI COMMANDS

```
aws sqs get-queue-attributes --queue-url <url> --attribute-names All                          # Check queue attributes
aws sqs get-queue-url --queue-name <name>                                                     # Get queue URL
aws sqs get-queue-attributes --queue-url <url> --attribute-names ApproximateNumberOfMessages  # Check message count
aws sqs receive-message --queue-url <url> --max-number-of-messages 10 --visibility-timeout 30 # Receive messages
aws sqs delete-message --queue-url <url> --receipt-handle <receipt>                           # Delete message
aws sqs purge-queue --queue-url <url>                                                         # Clear all messages (use carefully!)
```

10 MESSAGE FLOW (IDEAL)

Balanced Rate | Fast Processing | Empty Queue

KEY TAKEAWAYS

- ✓ Keep producers and consumers in balance.
- ✓ Set visibility timeout longer than processing time.
- ✓ Use DLQ to isolate bad messages.
- ✓ Scale consumers based on load.
- ✓ Monitor backlog age -- it tells the real story.

PRO TIP

Alert on ApproximateAgeOfOldestMessage

It's the canary for backlog!

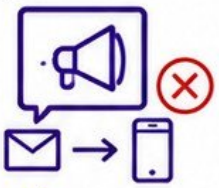
REMEMBER

- ☐ Backlog grows silently.
- ☐ Alerts should fire early.
- ☐ Fix rate, timeout, or failures.

INCIDENT #25

VERIQTA

SNS NOTIFICATIONS NEVER ARRIVE



Messages are published to SNS topics but subscribers never receive them.
Root cause is usually policy, subscription, or delivery issues.

AT A GLANCE

- Verify topic policies and permissions
- Ensure subscriptions are confirmed
- Check delivery status and failures
- Use CloudWatch metrics and logs
- Test with known-good endpoints

1 WHAT DOES IT MEAN?

SNS accepts the message but one or more subscribers don't receive the notification.

You may see:

- No emails / SMS / push received
- HTTP endpoints not called
- No errors in publish response
- Confusing or no delivery logs

Published but not delivered

2 TOPIC POLICIES

Incorrect or restrictive topic policies block delivery.

Check:

- Allow SNS to publish
- Allow target service (S3, Lambda, SQS, etc.) to subscribe
- No Deny statements blocking principals
- Cross-account access configured

POLICY EXAMPLE (ALLOW)

```
{
"Effect": "Allow",
"Principal": {"Service": "sns.amazonaws.com"},
"Action": "sns:Publish",
"Resource": "arn:aws:sns:us-east-1:123456789012:MyTopic"
}
```

3 SUBSCRIPTION CONFIRMATION

Many protocols require manual confirmation before delivery starts.

Check:

- Email – confirm the link
- SMS – opt-in required
- HTTP/HTTPS – confirm endpoint (200 OK required)
- Lambda – permissions to invoke required

SUBSCRIPTION STATUS

Confirmed	→ Messages delivered
Pending	→ Awaiting confirmation
Deleted	→ Will not receive

4 DELIVERY FAILURES

SNS tried to deliver but failed.

Common causes:

- Invalid email / phone / endpoint
- Endpoint unreachable
- HTTP 4xx/5xx responses
- Missing IAM permissions
- Blocked by network / firewall
- Message size limits exceeded

5 MONITORING & LOGS

Use CloudWatch metrics and logs to identify issues.

Key Metrics:

- NumberOfMessagesPublished
- NumberOfNotificationsDelivered
- NumberOfNotificationsFailed
- DeliveryFailure – metric
- Delivery status in CloudWatch Logs

6 COMMON CAUSES

- Subscription not confirmed
- Topic policy denies publish/subscribe
- Incorrect endpoint (typo / invalid)
- Endpoint returns error responses
- Lambda permission to invoke missing
- SQS queue policy blocks SNS
- SNS in one region, endpoint in another
- Message size > 256 KB (SQS/SNS limit)
- Opt-in not enabled for SMS

Most issues are configuration, not AWS service failures.

7 TROUBLESHOOTING STEPS

1. Check subscription status (Confirmed?)
2. Confirm email/SMS subscriptions
3. Verify topic policy allows access
4. Validate endpoint URL / format
5. Test endpoint manually (curl/postman)
6. Check delivery status in CloudWatch Logs
7. Review IAM permissions
8. Check for endpoint errors (4xx/5xx)
9. Test with AWS CLI publish
10. Fix and retest

8 USEFUL AWS CLI COMMANDS

```
# List topics
aws sns list-topics

# List subscriptions
aws sns list-subscriptions-by-topic \
  --topic-arn arn:aws:sns:REG:ACC:MyTopic

# Get subscription attributes
aws sns get-subscription-attributes \
  --subscription-arn <sub-arn>

# Publish test message
aws sns publish --topic-arn <topic-arn> \
  --message "Test message"

# Check delivery status in logs
aws logs filter-log-events \
  --log-group-name /aws/sns/*
```

9 DELIVERY FLOW (HOW IT SHOULD WORK)

- Message published
- Subscription confirmed
- Delivery successful
- Subscriber receives it

10 ENDPOINT PROTOCOLS & NOTES

PROTOCOL	CONFIRMATION	NOTES
Email	Required	Check inbox & spam folder
SMS	Required (Opt-in)	Phone number must be valid
HTTP/HTTPS	Optional (200 OK)	Endpoint must return 200 OK
Lambda	No	SNS needs lambda:InvokeFunction
SQS	No	Queue policy must allow SNS
Application	No	Client SDK receives the message

TIP

Always test with a simple email subscription first. It's the fastest way to confirm your setup!

KEY TAKEAWAYS

- ✓ Policies control who can publish and deliver.
- ✓ Confirm subscriptions before expecting messages.
- ✓ Delivery issues are visible in metrics and logs.
- ✓ Test early with known-good endpoints.
- ✓ Monitoring and alerts prevent silent failures.

> SNS doesn't fail silently.
> It's trying.
> It's just being blocked.

REMEMBER

- ☐ Check policy.
- ☐ Confirm subscription.
- ☐ Check delivery.
- ☐ Monitor & alert.

INCIDENT #26

VERIQTA

KMS KEY ACCESS DENIED

Applications or services cannot use an AWS KMS key for encryption or decryption.
Root cause is usually key policy, IAM permissions, or cross-account configuration.

AT A GLANCE

- Verify key policy and IAM permissions
- Check principal, action, and resource
- Review cross-account access setup
- Validate encryption context
- Use CloudTrail logs to troubleshoot

1 WHAT DOES IT MEAN?

A request was made to use a KMS key, but KMS denied the request with an AccessDeniedException.

Common errors:

- AccessDeniedException
- Not authorized to perform kms:Decrypt
- User is not authorized to perform kms:Encrypt
- KMS.NotFoundException (wrong key ID)

2 KEY POLICIES

The key policy defines who can use the key and what they can do.

Check:

- Principal is allowed
- Action (Encrypt/Decrypt/GenerateDataKey)
- Resource is correct (key ARN)
- Conditions (kms:EncryptionContext, aws:SourceAccount, aws:SourceArn)

KEY POLICY EXAMPLE (ALLOW)

```
{
"Effect": "Allow",
"Principal": { "AWS": "arn:aws:iam::123456789012:role/AppRole"},
"Action": ["kms:Decrypt", "kms:Encrypt", "kms:GenerateDataKey*"],
"Resource": "*",
"Condition": {"StringEquals": {
    "aws:SourceAccount": "123456789012"}}
}
```

3 IAM PERMISSIONS

IAM policies must allow the requested KMS actions on the key.

Check:

- IAM user/role has kms:Encrypt / kms:Decrypt / kms:GenerateDataKey*
- Resource is the key ARN
- No explicit DENY in IAM policy
- Session policy / permission boundary not restricting

IAM POLICY EXAMPLE (ALLOW)

```
{
"Effect": "Allow",
"Action": ["kms:Encrypt", "kms:Decrypt",
 "kms:GenerateDataKey*"],
"Resource": "arn:aws:kms:us-east-1:
123456789012:key/abcd-1234-efgh-5678"
}
```

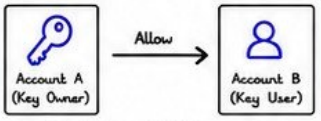
4 CROSS-ACCOUNT ACCESS

To use a key from another AWS account, both key policy and IAM must allow it.

Check:

- Key policy allows the external account
- Include AWS account ID or role ARN
- IAM role in other account has permission
- Trust relationship configured correctly
- Use External ID if required

CROSS-ACCOUNT FLOW



Key Policy + IAM Permissions

5 ENCRYPTION CONTEXT

If the key policy has conditions on encryption context, mismatches will cause access denied.

Check:

- Application sends correct context
- Keys and values match exactly
- Case-sensitive
- Extra or missing keys will fail

Example Condition:

```
"Condition": {
  "StringEquals": {
 "kms:EncryptionContext:app": "billing" }}
```

6 COMMON CAUSES

- Missing or incorrect key policy
- IAM policy missing KMS permissions
- Explicit DENY in IAM or key policy
- Wrong key ARN or alias
- Cross-account access not allowed
- Encryption context mismatch
- Service not authorized to use key
- Key disabled, scheduled for deletion, or in wrong region

Key policy is the boss.
IAM policy is the employee.

7 TROUBLESHOOTING STEPS

1. Check full error message and request ID
2. Verify key ARN / alias and region
3. Review key policy (is principal allowed?)
4. Check IAM policy (has required actions?)
5. Look for explicit DENY
6. Validate encryption context
7. For cross-account: confirm both sides
8. Check key state (enabled, not deleted)
9. Check CloudTrail for denied events
10. Test with AWS CLI

8 USEFUL AWS CLI COMMANDS

```
# Describe key
aws kms describe-key \
  --key-id <key-id-or-alias>

# List key policies
aws kms get-key-policy \
  --key-id <key-id-or-alias> \
  --policy-name default

# Simulate a principal policy
aws iam simulate-principal-policy \
--policy-source-arn arn:aws:iam::123456789012:role/AppRole \
--action-names kms:Encrypt kms:Decrypt \
--resource-arns arn:aws:kms:us-east-1:
  123456789012:key/abcd-1234-efgh-5678
```

9 HELPFUL CLOUDTRAIL CLUES

Look for KMS events with errorCode = AccessDenied.

EVENT NAME	ERROR CODE	WHAT IT MEANS
Decrypt	AccessDenied	Not allowed to decrypt
Encrypt	AccessDenied	Not allowed to encrypt
GenerateDataKey	AccessDenied	Not allowed to generate data key

Filter by eventSource: kms.amazonaws.com
Review userIdentity, errorMessage, and requestParameters.

10 KMS ACCESS FLOW (IDEAL)

ALL MUST BE ALLOWED

- IAM allows the action
- Key policy allows the principal
- No explicit DENY
- Conditions match

TIP

78% of KMS access denied errors are due to policy or permission issues, not KMS itself. CloudTrail is your best friend.

KEY TAKEAWAYS

- ✓ KMS is strict by design—policies control access.
- ✓ Key policy + IAM policy = Access. Both must allow.
- ✓ Cross-account access requires setup on both sides.
- ✓ Encryption context must match exactly.
- ✓ Test, validate, and least-privilege always.

> The right key,
> in the right hands,
> with the right permissions.

REMEMBER

- ☐ Policies control access.
- ☐ Principals need permission.
- ☐ KMS logs tell the truth.
- ☐ Verify, test, and secure.

INCIDENT #27

VERIQTA

ELASTICACHE PERFORMANCE DEGRADATION

Applications become slow due to high latency, high CPU, high memory usage, or cache inefficiency.
Root cause is usually cache misses, evictions, memory pressure, or under-provisioned cache.

AT A GLANCE

- Identify the performance bottleneck
- Improve cache hit ratio
- Fix memory pressure and evictions
- Right-size and scale correctly
- Monitor and prevent future issues

1 WHAT DOES IT MEAN?

ElastiCache is slower than expected.

You may see:

- High latency in application
- High CPU usage on cache nodes
- High memory usage
- Evictions happening
- More cache misses
- Increased backend (DB) load

User → App → ElastiCache

Slow cache = Slow application

2 COMMON CAUSES

- Cache miss rate is high
- Memory is full (evictions)
- Insufficient cache node size
- Too many connections
- Large keys or values
- Inefficient data patterns
- Wrong TTL settings
- Network latency
- Hot keys (uneven access)

CACHE METRICS TO CHECK

- CurrConnections
- CPUUtilization
- DatabaseMemoryUsage%
- CacheHitRate
- Evictions
- ReplicationLag (if cluster mode)

3 CACHE MISS VS HIT

CACHE HIT

Request → Cache → Fast

CACHE MISS

Request → Cache → DB → Slow

Remember:
High miss rate = More DB load
More DB load = Slow app
Keep your cache hot and efficient!

4 MEMORY PRESSURE & EVICTIONS

When cache memory is full, ElastiCache evicts old keys.

Check:

- Evictions increasing
- DatabaseMemoryUsage% ~ 100%
- FreeableMemory ~ 0
- CurrItems increasing too fast

EVICTION POLICY (Redis)

- allkeys-lru (default)
- volatile-lru (recommended)
- allkeys-random
- noeviction (avoid this)

5 HOW TO IMPROVE PERFORMANCE

- Increase node size (memory/CPU)
- Enable Cluster Mode (scale out)
- Use read replicas
- Optimize TTL (not too short/long)
- Avoid caching huge objects
- Use consistent data patterns
- Cache only what matters (hot data)

Small → Bigger → Faster!

6 SCALING STRATEGY

Vertical Scaling
Increase node size
(More memory/CPU)

Horizontal Scaling
Enable Cluster Mode
(Scale out)

Read Replicas
Offload read operations

Auto Scaling (Coming Soon)
Plan & Monitor

7 TROUBLESHOOTING STEPS

1. Check CacheHitRate (should be >80%)
2. Check CPUUtilization (<70% ideally)
3. Check DatabaseMemoryUsage% (<85%)
4. Check Evictions (should be low)
5. Review slow logs / latency
6. Analyze hot keys (use redis-cli --hotkeys)
7. Right-size or enable Cluster Mode
8. Optimize TTL and data patterns

QUICK CLI CHECKS (Redis)

```
> info stats | grep keyspace_hits
> info stats | grep keyspace_misses
> info memory
> redis-cli --hotkeys
```

8 USEFUL CLOUDWATCH METRICS

- CPUUtilization
- DatabaseMemoryUsagePercentage
- CurrConnections
- CacheHitRate
- Evictions
- NetworkBytesIn / Out
- ReplicationLag (for replicas)

9 COMMON SCENARIOS

- Traffic spike → Cache overloaded
- Low TTL → More misses
- Large value objects → Memory full
- Hot keys → Uneven load
- Small node → Not enough memory

10 OPTIMIZED ARCHITECTURE (IDEAL)

Users → Application → ElastiCache Cluster Mode → Database

High Hit Rate | Low Latency | Less DB Load | Better Performance

11 BEST PRACTICES

- Keep CacheHitRate high (>80%)
- Monitor memory and evictions
- Right-size your nodes
- Use Cluster Mode for scale
- Set proper TTL
- Avoid caching sensitive/large data

KEY TAKEAWAYS

- ✓ High performance = High hit rate + Right sizing
- ✓ Monitor memory, CPU, and evictions
- ✓ Optimize TTL and data patterns
- ✓ Scale before users feel the pain
- ✓ Cache smart, not everything!

> A fast cache is invisible.
> A slow cache is painful.
> Make your cache a superpower, not a bottleneck.

REMEMBER

- ☐ Monitor continuously.
- ☐ Act early.
- ☐ Optimize regularly.
- ☐ Users don't care – they just want speed!

INCIDENT #28

VERIQTA

AWS CONFIG DETECTS MASSIVE CONFIGURATION DRIFT

AWS Config detects that resources are not matching the desired configuration.
Root cause is usually manual changes, missing controls, or lack of enforcement.

AT A GLANCE

- Analyze and understand the drift
- Evaluate compliance impact
- Investigate resource history
- Remediate and enforce compliance
- Automate and prevent future drift

1 WHAT DOES IT MEAN?

AWS Config compares actual resource configuration with the recorded (desired) configuration.

When they don't match, you get a configuration drift.

You may see:

- Config drift notifications
- Compliance rule NON_COMPLIANT
- Unexpected resource changes
- Security policy violations
- Cost or performance impact

Desired Configuration → AWS Config Check → Drift Detected

2 DRIFT ANALYSIS

Identify what changed, when, and who (or what) made the change.

Check:

- Which resources are drifting
- What properties changed
- When the change occurred
- Who/what triggered the change
- Impact on security, cost, or availability

DRIFT STATUS

IN_SYNC | DRIFTED | NOT EVALUATED

3 COMPLIANCE RULES

Config rules evaluate resources against best practices and policies.

Check:

- Which rules are NON_COMPLIANT
- Why the resource failed
- Regulatory or internal policies
- Severity and risk level
- Aggregate compliance across accounts/regions

RULE EXAMPLE

Rule: s3-bucket-public-read-prohibited
Status: NON_COMPLIANT
Resource: my-public-bucket
Reason: Bucket has public read access

4 RESOURCE HISTORY

Review timeline to understand how drift happened.

Check:

- Configuration timeline
- Past and current values
- Change origin (Console, CLI, API, CloudFormation, etc.)
- Associated events and users
- Relationships with other resources

TIMELINE VIEW

Past → Change → Now

5 REMEDIATION WORKFLOW

Fix the drift and bring resources back to the desired state.

Steps:

1. Confirm the desired configuration
2. Decide: rollback or accept change
3. Apply remediation (manual or auto)
4. Re-evaluate with AWS Config
5. Validate compliance

Fix → Apply → Compliant

6 COMMON CAUSES

- Manual changes in Console/CLI
- Missing or weak guardrails
- Untracked infrastructure
- Failed automation or pipelines
- CloudFormation stack drift
- Third-party or external changes
- Lack of monitoring or alerts
- Policy changes not enforced

Prevent drift with guardrails, automation, and visibility.

7 TROUBLESHOOTING STEPS

1. Identify drifting resources (Config)
2. Review rule and non-compliance reason
3. Check resource configuration history
4. Verify change origin and intent
5. Compare with IaC / desired state
6. Fix or revert the change
7. Re-run Config evaluation
8. Confirm compliance restored
9. Update policies/automation if needed
10. Monitor for re-occurrence

8 USEFUL AWS CLI COMMANDS

```
aws configservice describe-compliance-by-config-rule --config-rule-name RULE_NAME   # Check rule
aws configservice get-compliance-details-by-config-rule --config-rule-name RULE_NAME   # Rule details
aws configservice describe-config-resources --config-rule-name RULE_NAME   # Drifting resources
aws configservice get-resource-config-history --resource-type TYPE --resource-id ID   # Resource history
aws configservice start-config-rules-evaluation --config-rule-names RULE_NAME   # Re-evaluate rule
```

9 CONFIG DRIFT EVENT FLOW

Resource Change → AWS Config Detects Drift → Config Rule Evaluates → NON_COMPLIANT (Drift Found) → Alert Triggered (SNS / CloudWatch) → Engineer Investigates → Remediate → Compliance Restored

Detect → Analyze → Remediate → Enforce → Prevent

10 BEST PRACTICES

- Enable AWS Config in all regions
- Record all resource types
- Use managed + custom rules
- Alert on NON_COMPLIANT changes
- Enforce with SCPs, Config Rules & IaC
- Automate remediation with SSM / Lambda
- Review Config snapshots regularly

TIP

Don't just detect drift—enforce compliance. Use automation and guardrails so drift is rare, not just remediated.

KEY TAKEAWAYS

- ✓ AWS Config tells you what changed.
- ✓ Analysis shows why and when it happened.
- ✓ Remediate, enforce, and automate.
- ✓ Drift is normal—leaving it unaddressed is the risk.

> Good teams detect drift.
> Great teams prevent it.
> — Build guardrails, not just alerts.

REMEMBER

- ☐ Detect early.
- ☐ Understand impact.
- ☐ Remediate quickly.
- ☐ Automate compliance.

INCIDENT #29

VERIQTA

CLOUDTRAIL MISSING CRITICAL EVENTS

CloudTrail is your audit trail for AWS API activity. Missing events create blind spots and security risk. Root cause is usually misconfiguration, delivery failure, or integrity issues.

AT A GLANCE

- Ensure complete and reliable API activity logging
- Capture events in all Regions
- Verify S3 delivery is working
- Protect log integrity
- Review and monitor regularly

1 WHAT DOES IT MEAN?

CloudTrail is enabled but critical API events are missing from logs.

You may see:

- Gaps in event history
- Missing API actions in logs
- Inconsistent audit records
- Troubleshoot & security investigations impacted
- Compliance failures

AWS Activity → CloudTrail → Missing Events

2 TRAIL CONFIGURATION

Incorrect or incomplete Trail setup.

Check:

- Is CloudTrail enabled?
- Is the event selector capturing management + data events?
- Are Insights enabled?
- Is the Trail in ACTIVE status?
- Is logging for all regions enabled?

RECOMMENDED SETTINGS

- Include Global service events
- Include Read & Write (data events)
- Enable CloudTrail Insights
- Multi-Region Trail

3 MULTI-REGION LOGGING

Logging in only one region causes events from other regions to be missed.

Check:

- Is Multi-Region Trail enabled?
- Are all regions sending logs to the same S3 bucket?
- Verify event history across all regions

GOOD PRACTICE

Enable Multi-Region Trail and use a central S3 bucket for all regions.

4 S3 DELIVERY

Delivery failures or misconfigured S3 bucket can cause missing logs.

Check:

- S3 bucket exists and is correct
- Bucket policy allows CloudTrail
- KMS permissions (if encrypted)
- S3 bucket not full or blocked
- Check CloudWatch Metrics for delivery errors

DELIVERY FLOW

CloudTrail → S3 Bucket → Delivered

5 LOG INTEGRITY

Log tampering or deletion breaks audit trust.

Check:

- Log file validation enabled?
- S3 Object Lock (WORM) enabled?
- KMS CMK in use (encryption)?
- CloudTrail digest validation automated?

INTEGRITY CONTROLS

S3 Object Lock | KMS Encryption | Log File Validation

6 COMMON CAUSES

- Trail not enabled in all regions
- Incorrect event selector
- S3 bucket policy misconfigured
- KMS permission issues
- S3 delivery failures
- Bucket deleted or renamed
- Log file validation disabled
- CloudTrail service issues (rare)

Any of these can lead to missing critical events.

7 TROUBLESHOOTING STEPS

1. Check if CloudTrail is Enabled
2. Validate Trail status = ACTIVE
3. Check event selector settings
4. Verify Multi-Region Trail
5. Check recent CloudWatch metrics (DeliveryErrors)
6. Inspect S3 bucket permissions
7. Validate log file integrity
8. Review AWS Health Dashboard

USEFUL TOOLS

- CloudTrail Lookup Events
- CloudTrail Lake
- CloudWatch Metrics

8 USEFUL CLI COMMANDS

```
# List trails
aws cloudtrail describe-trails
# Get trail status
aws cloudtrail get-trail-status --name <trail-name>
# Lookup events
aws cloudtrail lookup-events\
  --lookup-attributes AttributeKey=EventName,
    AttributeValue=ConsoleLogin
# Describe S3 delivery
aws s3api get-bucket-policy \
  --bucket <bucket-name>
# Describe CloudWatch metrics
aws cloudwatch get-metric-statistics \
  --namespace AWS/CloudTrail \
  --metric-name DeliveryErrors
```

9 EVENT FLOW (IDEAL)

AWS Resource → AWS Activity (API Call) → CloudTrail Records → S3 Bucket (Central) → Log Integrity (Validated) → Audit & Monitoring

Capture → Store → Protect → Monitor → Investigate

10 BEST PRACTICES

- Enable Multi-Region Trail
- Log all Management events
- Include Read/Write Data events
- Store logs in dedicated S3 bucket
- Enable log file validation
- Use S3 Object Lock (WORM)
- Monitor CloudWatch metrics & alerts
- Review regularly

TIP

If you can't see it in CloudTrail, it's like it never happened. Complete logs = Complete visibility.

KEY TAKEAWAYS

- ✓ Correct configuration is critical for complete logging.
- ✓ Multi-Region Trail prevents missing events.
- ✓ Secure delivery and integrity protect your audit trail.
- ✓ Monitor, validate, and review CloudTrail regularly.

> No logs, no visibility.
> No visibility, no security.
> Make CloudTrail complete.

REMEMBER

- ☐ Configure correctly.
- ☐ Capture everything.
- ☐ Protect your logs.
- ☐ Monitor continuously.

INCIDENT #30

VERIQTA

CROSS-REGION DISASTER RECOVERY FAILOVER

Your primary region is down or unhealthy.
We failover to the DR region to restore service.
Success depends on workflows, DNS routing, data health, and validation.

AT A GLANCE

- Follow DR runbook, step by step
- Route traffic using Route 53 failover
- Verify data replication and freshness
- Validate applications and dependencies
- Communicate and document everything

1 WHEN TO FAILOVER?

Failover if the primary region has:

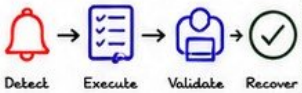
- Regional outage or major incident
- Critical services unhealthy
- SLA / error budget at risk
- Databases unavailable
- Long recovery ETA

Remember:

Fail fast,
recover faster.
Communication is critical!

2 RECOVERY WORKFLOW

1. Detect & confirm outage
2. Declare failover (War Room)
3. Isolate primary region (if needed)
4. Failover DNS (Route 53)
5. Start services in DR region
6. Validate application + data
7. Monitor closely
8. Communicate to all stakeholders
9. Plan failback (when stable)



3 ROUTE 53 FAILOVER

Use Route 53 Failover Routing Policy.
Two records: Primary & Secondary.

How it works:

- Health check monitors primary endpoint
- If unhealthy, Route 53 routes to DR
- TTL should be low (30–60s)
- Test failover regularly!

FAILOVER FLOW

4 DATA REPLICATION

Ensure data in DR region is fresh and consistent.

Common replication options:

- RDS Multi-AZ / Read Replica (Cross-Region)
- Aurora Global Database
- DynamoDB Global Tables
- S3 Cross-Region Replication
- EBS Snapshot Copy
- EFS Replication (AWS DataSync)
- Kinesis Data Streams (Enhanced Fan-Out)

CHECK DATA HEALTH

- Replication lag
- Last update time
- Consistency check
- Integrity validation

5 VALIDATION CHECKLIST

- [ ] DNS is routing to DR region
- [ ] Load balancer healthy
- [ ] Application endpoints responding
- [ ] Database connections successful
- [ ] Data integrity verified
- [ ] Critical user flows tested
- [ ] Background jobs running
- [ ] Alerts & dashboards active
- [ ] Synthetic tests passing
- [ ] Stakeholders notified

6 COMMUNICATION PLAN

- Declare incident & failover
- Provide status page updates
- Notify stakeholders / customers
- Update internal channels
- Share ETA for recovery
- Document decisions & timelines

COMMUNICATION TEMPLATE

Subject: DR Failover Activated
We have failed over to the DR region.
Impact: <brief description>
Actions taken: <summary>
Next update in <time>.

7 MONITORING IN DR REGION

After failover, watch these closely:

- CPU, Memory, Disk, Network
- Application Latency & Errors
- Database Connections / Replication Lag
- Queue Depths & Throughput
- Cost & Resource Utilization
- User Experience (Real User Monitoring)

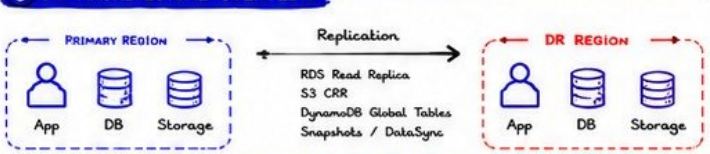
USEFUL TOOLS

- CloudWatch Dashboards
- CloudWatch Alarms
- X-Ray / Datadog / New Relic
- Synthetic Monitoring

8 FAILBACK (WHEN STABLE)

1. Ensure primary region is healthy
2. Sync latest data back (if needed)
3. Update Route 53 to Primary
4. Validate services in primary region
5. Monitor for stability
6. Decommission DR resources (optional)
7. Post-incident review & lessons learned

9 DR ARCHITECTURE OVERVIEW



High Availability • Automation • Testing • Monitoring • Documentation

10 BEST PRACTICES

- Test DR regularly (at least quarterly)
- Automate infra with IaC (CloudFormation/Terraform)
- Keep runbooks updated
- Use Route 53 health checks
- Keep backups and snapshots
- Monitor replication lag
- Document RTO, RPO, and dependencies
- Review and improve after every drill

TIP

A DR plan you didn't test
is just a document.
Test today,
survive tomorrow.

KEY TAKEAWAYS

- ✓ Have a tested DR plan and runbook.
- ✓ Fast DNS failover restores user access.
- ✓ Healthy, replicated data is critical.
- ✓ Validate everything before declaring success.
- ✓ Communicate early and often.

"Disasters happen.
Prepared teams recover.
Prepared systems survive."

REMEMBER

- [ ] Plan for failure.
- [ ] Automate recovery.
- [ ] Validate continuously.
- [ ] Learn and improve always.

INCIDENT #17

VERIQTA

ENTERPRISE AWS INCIDENT RESPONSE WORKFLOW

A structured response minimizes impact, reduces MTTR, and ensures clear communication and accountability. Root cause is usually detection gaps, unclear process, or slow communication.

AT A GLANCE

- Detect early, act fast
- Assess impact & severity
- Escalate to the right people
- Communicate clearly
- Recover safely & validate
- Document & learn

1 INCIDENT SEVERITY LEVELS

SEVERITY	IMPACT	EXAMPLES	RESPONSE
P0	Critical (Down)	Whole system down, major outages	< 15 min response
P1	High (Severe)	Partial outage, major functionality affected	< 30 min response
P2	Medium (Degraded)	Performance issues, non-critical errors	< 2 hours response
P3	Low (Minor)	Minor issues, cosmetic flaws	< 1 day response

Severity can CHANGE as more information becomes available.

2 ESCALATION PROCESS

1. **Detect & Alert** — Monitoring / Alertmanager. / AWS Health
2. **Triage** — On-call Engineer analyzes & assesses severity
3. **Escalate** — Notify team lead / manager based on severity
4. **Incident Commander** — Assigned based on impact (Owns the incident)
5. **War Room Activated** — Cross-functional team joins (Comms, Dev, Ops, SRE, etc.)

3 COMMUNICATION FLOW

Status Updates

Every 15-30 mins or when significant change occurs

Communication Channels

- Slack / Microsoft Teams
- Email (Executive Updates)
- Status Page (User Updates)
- Conference Bridge (War Room)

4 RECOVERY TIMELINE (PHASES)

- **0-15 min — DETECT & ALERT**: Alert triggered, incident created, initial assessment.
- **15-30 min — ASSESS & ESCALATE**: Triage complete, severity determined, escalation done.
- **30-60 min — MITIGATE**: Stop the impact, apply temporary fix / workaround.
- **1-4 hrs — RESOLVE**: Implement permanent fix, restore full functionality.
- **4+ hrs — VALIDATE & CLOSE**: Validate systems, close incident, share post-incident summary.

5 INCIDENT COMMAND PRINCIPLES

- One incident, one commander
- Clear ownership & decisions
- Timely communication
- Focus on impact & users
- Document everything
- Blameless culture
- Learn & improve

6 KEY ROLES & RESPONSIBILITIES

- Incident Commander: Overall ownership
- SRE / Ops: Infrastructure & mitigation
- Developers: Code / application fixes
- QA / Validation: Testing & verification
- Communications Lead: Internal & external updates
- Product Owner / Business: Impact assessment
- Executive Sponsor: High-level decisions

7 RESPONSE CHECKLIST

- [x] Alert acknowledged
- [x] Incident created (ticket)
- [x] Severity assessed
- [x] Stakeholders notified
- [x] War room started
- [x] Impact identified
- [x] Mitigation plan executed
- [x] Customers updated
- [x] Root cause analysis started
- [x] Post-incident review scheduled

8 USEFUL AWS TOOLS

CloudWatch	Monitoring & Alarms
AWS Health	AWS Service health
CloudTrail	Audit & Activity logs
Systems Manager	Operations & Automation
Route 53	DNS Failover
Chatbot	Alert to Slack/Teams
EventBridge	Event-driven response

9 INCIDENT RESPONSE FLOW (HIGH LEVEL)

Speed + Communication + Ownership = Successful Recovery

10 POST-INCIDENT ACTIVITIES

- Conduct blameless post-mortem
- Identify root cause & contributing factors
- Update runbooks & documentation
- Improve monitoring & alerting
- Implement preventive actions
- Share lessons learned across teams
- Track action items to completion

TIP

The best response is practiced before you need it. Run disaster drills regularly.

KEY TAKEAWAYS

- ✓ Prepare before incidents happen.
- ✓ Communicate early and often.
- ✓ Follow the workflow, not emotions.
- ✓ Document and learn to prevent repeats.

> "In incident response, process is your best weapon and communication is your shield."

REMEMBER

- ☐ People first, then systems.
- ☐ Keep stakeholders informed.
- ☐ Restore service, then optimize.
- ☐ Every incident is a chance to improve.

INCIDENT #18

VERIQTA

AWS WELL-ARCHITECTED PRODUCTION CHECKLIST

Use this checklist to validate that your AWS workloads follow Well-Architected best practices in production. Review regularly and improve continuously.

AT A GLANCE

- Validate your architecture
- Reduce risks and failures
- Improve performance
- Optimize costs
- Build secure & reliable systems
- Operate with excellence

1 OPERATIONAL EXCELLENCE

- Define and document processes
- Automate operations
- Use Infrastructure as Code
- Implement CI/CD pipelines
- Monitor and alert proactively
- Runbooks and playbooks ready
- Track metrics and logs
- Post-incident reviews (PIR)
- Continuously improve

KEY FOCUS

Run and monitor systems effectively and improve operations over time.

2 SECURITY

- Use least privilege IAM
- Enable MFA for all users
- Encrypt data in transit (TLS)
- Encrypt data at rest
- Use AWS KMS for key mgmt
- Enable CloudTrail (all regions)
- Enable GuardDuty & Security Hub
- Keep systems patched
- Perform regular vulnerability scans
- Implement network segmentation
- Review and audit access

KEY FOCUS

Protect systems, data, and access across the entire environment.

3 RELIABILITY

- Design for fault tolerance
- Use Multi-AZ / Multi-Region
- Enable automated backups
- Test backup restores regularly
- Use Auto Scaling groups
- Health checks & self-healing
- Use proven AWS services
- Reduce manual interventions
- Plan and test disaster recovery

KEY FOCUS

Ensure workloads are available, resilient, and recoverable.

4 PERFORMANCE EFFICIENCY

- Use right-size instances
- Use Auto Scaling
- Load balance traffic
- Use caching (ElastiCache / CDN)
- Optimize database performance
- Use async processing (SQS, SNS)
- Monitor performance metrics
- Run load & stress tests
- Optimize APIs and queries

KEY FOCUS

Use resources efficiently to meet performance requirements.

5 COST OPTIMIZATION

- Use Cost Explorer & Budgets
- Tag resources consistently
- Right-size resources
- Use Savings Plans / Reserved Instances
- Schedule non-prod environments
- Use Spot Instances where possible
- Delete unused resources
- Optimize storage classes
- Review and optimize continuously

KEY FOCUS

Deliver business value at the lowest possible cost.

6 SUSTAINABILITY

- Understand your workload impact
- Use efficient AWS services
- Right-size resources
- Reduce over-provisioning
- Use managed services
- Optimize data transfer
- Use low-carbon Regions (when possible)
- Set sustainability goals & track
- Improve continuously

KEY FOCUS

Minimize environmental impact and build for the future.

WELL-ARCHITECTED REVIEW CADENCE

Quarterly Review → Validate Checklist → Track Improvements → Continuous Optimization

TIP

Well-Architected is not a one-time activity.
Build it into your culture.
Small improvements every week = Big results.

KEY TAKEAWAYS

- ✓ Follow the 6 pillars for strong systems
- ✓ Review checklist regularly
- ✓ Automate, monitor, and measure
- ✓ Optimize cost and performance together
- ✓ Build secure, reliable, and sustainable systems

> "Well-Architected is a mindset. Excellence is a habit."

REMEMBER

- ☐ Validate always
- ☐ Monitor everything
- ☐ Automate where possible
- ☐ Improve continuously
- ☐ Customer value first

INCIDENT #19

VERIQTA

SENIOR AWS SYSTEM DESIGN INTERVIEW SCENARIOS

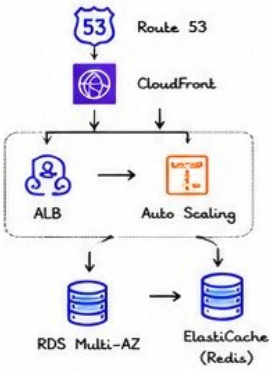
These are real architecture questions asked in Senior/Staff AWS interviews. Focus on clarity, trade-offs, and production-ready design.

AT A GLANCE

- Think out loud
- Clarify requirements
- Design for scale & HA
- Discuss trade-offs
- Justify your decisions
- Iterate and improve

1 DESIGN A HIGHLY AVAILABLE WEB APPLICATION

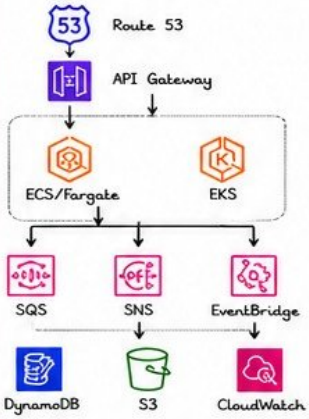
Requirements:
- Handle 10M requests/day
- 99.99% availability
- Static + dynamic content
- Global users



Key Points:
- Multi-AZ for DB
- Auto Scaling for compute
- CloudFront for global performance
- ElastiCache for caching

2 DESIGN A SCALABLE MICROSERVICES PLATFORM

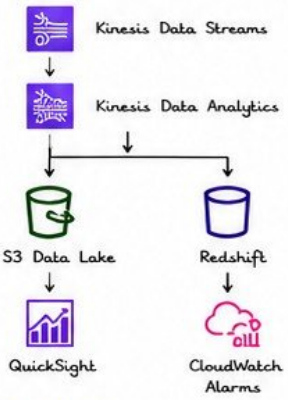
Requirements:
- 100+ microservices
- Independent deployments
- High availability
- Observability & security



Key Points:
- Container orchestration (ECS/EKS)
- Decouple with SQS/SNS
- Observability with CloudWatch/X-Ray
- Least privilege IAM & security groups

3 DESIGN A REAL-TIME ANALYTICS PLATFORM

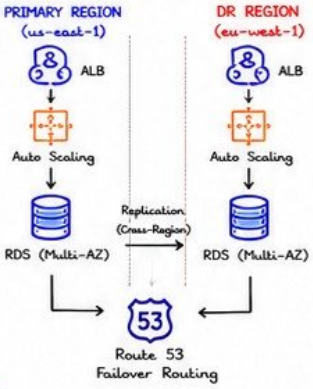
Requirements:
- Ingest millions of events/sec
- Real-time processing
- Store data for analytics
- Dashboards & alerts



Key Points:
- Kinesis for ingestion
- Real-time processing with KDA
- S3 + Redshift for analytics
- QuickSight for dashboards

4 DESIGN A MULTI-REGION DISASTER RECOVERY SOLUTION

Requirements:
- Primary in us-east-1
- DR in eu-west-1
- RPO < 15 min, RTO < 1 hour
- Automated failover



Key Points:
- Cross-region replication
- Route 53 failover
- Automated failover (Lambda)
- Regular DR testing

5 COMMON TRADE-OFFS

Decision	Option A	Option B	Trade-off
Compute	EC2	Fargate	Control vs. Managed
Database	RDS	DynamoDB	Consistency vs. Scale
Storage	EBS	S3	Block vs. Object
Caching	ElastiCache	DAX	Flexibility vs. Simplicity
Networking	NAT Gateway	VPC Endpoints	Cost vs. Complexity

6 SCALING DECISIONS

Vertical Scaling	Increase instance size	+ Simple - Downtime
Horizontal Scaling	Add more instances	+ High availability - More complexity
Auto Scaling	Scale based on metrics	+ Cost efficient - Needs tuning
Serverless Scaling	Scale to zero	+ Pay per use - Cold start

7 HIGH AVAILABILITY DISCUSSIONS

- Multi-AZ / Multi-Region
  Eliminate single points of failure
- Health Checks & Auto Recovery
  Detect & replace unhealthy resources
- Stateless Design
  Easy to scale and replace
- Data Durability
  Backups, replication, and versioning
- Testing & GameDays
  Chaos engineering & DR drills

8 SYSTEM DESIGN APPROACH

1. Clarify requirements → 2. Define scope → 3. High-level design → 4. Deep dive components → 5. Trade-offs & decisions → 6. Review & improve

"Communicate clearly. Justify every decision."

TIPS

- ✓ Ask clarifying questions
- ✓ Start high-level, then dive deep
- ✓ Think about failure scenarios
- ✓ Explain trade-offs clearly
- ✓ Always consider cost & ops

REMEMBER

- ☐ No perfect architecture
- ☐ Think in trade-offs
- ☐ Simplicity > Complexity
- ☐ Design for change
- ☐ Operate what you build

INCIDENT #20

VERIQTA

AWS PRODUCTION OPERATIONS CHEAT SHEET

Your quick reference guide for operating, troubleshooting, and managing AWS in production.

AT A GLANCE

- Know the services
- Follow the process
- Use the right tools
- Monitor everything
- Automate & improve
- Keep learning

1 CRITICAL AWS SERVICES

- **EC2** Compute
- **RDS** Databases
- **ELB / ALB** Load Balancing
- **Route 53** DNS
- **IAM** Access Management
- **Lambda** Serverless
- **DynamoDB** NoSQL Database
- **CloudTrail** Audit Logging
- **S3** Storage
- **ECS / EKS** Containers
- **CloudFront** CDN
- **CloudWatch** Monitoring
- **VPC** Networking
- **SQS / SNS** Messaging
- **AWS Backup** Backup

Know what each service does and when to use it.

2 INVESTIGATION SEQUENCE

Detect	Alert triggers or user reports an issue
Assess	Understand the impact & scope
Collect	Logs, metrics, events, traces, configs
Analyze	Identify root cause using evidence
Mitigate	Apply fix, workaround, or rollback
Validate	Confirm issue resolved & no side effects
Document	Record timeline, root cause, actions, lessons
Improve	Create action items & prevent recurrence

Follow the process. Don't guess. Investigate with data.

3 CLI ESSENTIALS

EC2

```
aws ec2 describe-instances --region us-east-1
aws ec2 start-instances --instance-ids i-123456
```

S3

```
aws s3 ls s3://your-bucket --region us-east-1
aws s3 cp file.txt s3://your-bucket/
```

CloudWatch

```
aws cloudwatch get-metric-statistics \
--namespace AWS/EC2 --metric-name CPUUtilization\
--dimensions Name=InstanceId,Value=i-123456 \
--start-time 2024-05-15T00:00:00Z \
--end-time 2024-05-15T01:00:00Z \
--period 300 --statistics Average
```

Logs

```
aws logs tail /aws/ec2/your-log-group \
--follow --region us-east-1
```

Systems Manager

```
aws ssm send-command --instance-ids i-123456 \
--document-name AWS-RunShellScript --parameters \
'commands=["uptime"]'
```

Use CLI for speed, automation & repeatability.

4 MONITORING CHECKLIST

- [ ] CloudWatch Alarms (CPU, Memory, Disk, 5XX)
- [ ] Application Logs (Errors, Latency, Timeouts)
- [ ] Infrastructure Logs (VPC Flow Logs, CloudTrail)
- [ ] Dashboard Health (Overview of key metrics)
- [ ] Synthetic Checks (Uptime & availability)
- [ ] Databases (Connections, CPU, IOPS, Locks)
- [ ] Capacity & Scaling (Auto Scaling, Limits)
- [ ] Cost Anomaly Alerts
- [ ] Security Alerts (GuardDuty, Security Hub)
- [ ] Backup Jobs & Restore Tests
- [ ] License / Certificate Expiry
- [ ] Third-party Integrations

KEY METRICS

Latency, Throughput, Error Rate, Saturation, Availability, Cost

5 ARCHITECTURE CHECKLIST

- [x] Multi-AZ / Multi-Region for critical components
- [x] Auto Scaling configured & tested
- [x] Load Balancers with health checks
- [x] RDS with backups, Multi-AZ & monitoring
- [x] S3 versioning, lifecycle & encryption.
- [x] IAM least privilege & MFA enabled
- [x] VPC design (Subnets, NACLs, SGs, Endpoints)
- [x] Encryption in transit (TLS) & at rest
- [x] CloudTrail enabled in all regions
- [x] Centralized logging & monitoring
- [x] Disaster recovery plan tested
- [x] Cost optimization reviewed

REMEMBER

A good architecture prevents incidents before they happen.

6 FINAL VERIQTA PRODUCTION PRINCIPLES

OBSERVE EVERYTHING
If you don't monitor it, you can't fix it.

AUTOMATE REPEATEDLY
Reduce human error and save time.

DESIGN FOR FAILURE
Failures will happen. Be prepared.

SECURE BY DEFAULT
Security is everyone's responsibility.

DOCUMENT RELENTLESSLY
Clear docs save hours in incidents.

IMPROVE CONTINUOUSLY
Postmortems drive real improvement.

FOCUS ON CUSTOMER IMPACT
Fix what matters most, always.

QUICK REFERENCE

- Check Status: status.aws.amazon.com
- Support Cases: console.aws.amazon.com/support
- Cost Explorer: Cost Explorer > Dashboard
- Trusted Advisor: Check > All Categories
- Architecture Icons: aws.amazon.com/architecture/icons/

PRO TIP

Before making any change in production:

- Understand the impact
- Test in lower environment
- Communicate with stakeholders
- Have a rollback plan

EMERGENCY CONTACT FLOW

Engineer → Incident Commander → Teams / Stakeholders

Communicate early.
Communicate clearly.
Communicate often.

"You can't prevent every incident, but you can prepare for every incident. Prepared teams win. Bootstrap teams lose.